LOCALDANMU

# 培训弹幕

## 实现文档 · 架构与模块说明

| | |
|---|---|
| 对应需求 | 《线下培训弹幕互动平台（自建私密版）需求方案 v1.0》 |
| 代码版本 | v0.1.6 |
| 代码仓库 | github.com/delaolao/LocalDanMu |
| 生成日期 | 2026 年 11 月 2 日 |

# 培训弹幕 · 实现文档

> 对应需求：**《线下培训弹幕互动平台（自建私密版）需求方案 v1.0》** 代码版本：`v0.1.3`（仓库 https://github.com/delaolao/LocalDanMu） 面向读者：接手本项目的系统架构师、前端/客户端工程师、运维同学

---

## 0. 怎么读这份文档

| 你的目的 | 建议阅读顺序 |
|---|---|
| 5 分钟了解这是什么、怎么跑起来 | §1 → §2 → §7 |
| 要改需求、加功能 | §2 → §4 → §5（对应机制）→ §12 |
| 要排查线上问题 | §5 → §10 → §9 |
| 要评估换技术栈 | §2 → §11 |

全文用 `文件:行号` 的方式指向源码，行号以 `v0.1.3` 为准，重构后可能漂移，以函数名为准。

---

## 1. 需求与约束

### 1.1 四条硬约束

需求方案里最重要的是四条"不可妥协"的约束，它们几乎决定了整个技术选型：

| 约束 | 含义 | 对实现的直接影响 |
|---|---|---|
| **不遮挡** | 弹幕要叠在 PPT 放映之上，且不能挡住讲师鼠标操作 | 必须有一个**透明、置顶、默认鼠标穿透**的全屏窗口；且要能在 WPS/PowerPoint 放映时仍然压在上面 |
| **不安装** | 观众不能装 App，扫码即用 | 观众端只能是普通 H5，跑在微信内置浏览器里；**不能依赖 HTTPS、不能依赖 Service Worker、不能用需要安全上下文的 API** |
| **私密** | 消息只在本机内存，不出局域网，默认不落盘 | 不能有云端服务、不能有第三方统计；HTTP 服务必须在本机跑；配置落盘要最小化并脱敏 |
| **仅 Windows** | 只支持 Windows 10/11 x64 | 允许使用 Windows 专有 API（置顶、点击穿透、user32 调用），不必为跨平台做抽象 |

### 1.2 需求确认阶段定下的关键决策

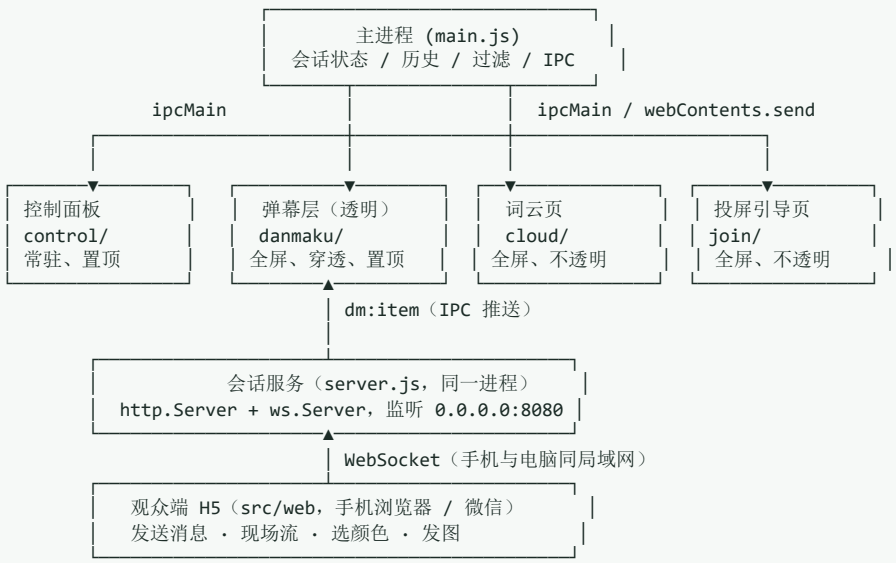
| 决策点 | 结论 | 原因 |
|---|---|---|
| 显示器选择 | 支持，弹幕层可指定投到第几块屏 | 讲师常是「笔记本 + 投影」双屏 |
| 审核后上屏 | **不做** | 现场培训要即时反馈，审核会打断节奏 |
| 房间口令 | 做（`/j/<8位token>`） | 防止隔壁会场/路人误入 |
| 观众端入口 | 微信扫码为主 | 现场最省事的动作是"扫一下" |
| 关键词过滤 | 5 条正则 + 内置 48 条通用脏话 + 自定义/导入 | 纯人工维护词库不现实；纯规则又不够 |

| 决策点 | 结论 | 原因 |
| --- | --- | --- |
| 词云切词 | 短句整条、长句按标点切 | 现场多为两三个字的短答 |
| 弹幕颜色 | 观众自选 8 色，默认随机 | 便于区分发言人，也照顾现场投影对比度 |

# 2. 总体架构

## 2.1 进程与窗口拓扑

Electron 的进程模型天然贴合"一主多窗"的需求：**主进程**负责会话、协议、过滤、存储；**每个窗口一个渲染进程**，互不影响。




要点：

- **服务端和主进程是同一个 Node 进程**，没有额外的后端服务，也没有数据库。
- 观众和讲师之间只有一条 WebSocket；大屏、词云、控制面板之间只有 IPC。
- 词云页与投屏引导页是**互斥的全屏窗口**：词云模式下弹幕层 `hide()`，避免两层抢焦点。

## 2.2 一条弹幕的完整旅程

```
手机输入
  │  ① ws.send({type:'message', contentType, payload, color, clientId})
  ▼
server.js onConnection
  │  ② validate()      结构/长度/图片 dataURL 白名单
  │  ③ filter()        5 条正则 + 48 条脏话；block 直接回 invalid，不占额度
  │  ④ 三道限流        单人间隔 → 单 IP 令牌桶 → 全局令牌桶
  │  ⑤ seq += 1
  ├─⑥ ack 回手机（seq / retryAfter）
  ├─⑦ broadcast {type:'item'} 给所有观众（图片只发占位）
  └─⑧ onMessage(out) → main.js handleIncoming
        │  ⑨ history.push（文本最多 300 条，图片最多 20 张，图片带缩略）
        ├─⑩ overlayWin.send('dm:item', item)   → 弹幕层入队
        ├─⑪ controlWin/cloudWin.send('dm:item', brief) → 最近弹幕列表 / 词云
        └─⑫ schedulePush() → 控制面板状态刷新（含在线人数）
```

# 3. 目录结构与文件职责

```
LocalDanMu/
├─ package.json                依赖、构建配置（nsis + portable）、版本号
├─ README.md                   使用者视角的说明（装、跑、排障）
├─ docs/architecture.md        本文档（architecture.pdf 是它的打印版）
├─ LocalDanMu-512x512.png      图标母版（换图标只改这张，再跑 make-icons.py）
├─ assets/                     生成出来的图标：icon.png / icon-256.png / tray-*.png
├─ scripts/                    构建、测试、发布、诊断脚本（不进安装包）
└─ src/
   ├─ main/                    主进程（Node 环境）
   ├─ renderer/                四个窗口的页面（浏览器环境）
   └─ web/                     观众端 H5（由 HTTP 服务直接吐出）
```

> 图标管线：`python scripts/make-icons.py` 读取根目录的 512 母版，按 LANCZOS 缩放出 `icon.png`（512，打包用）、`icon-256.png`、`tray-16/32.png`（托盘）。exe / 安装包 / 快捷方式的图标由 electron-builder 从 `icon.png` 生成 .ico，无需手工准备多尺寸。

## 3.1 主进程 `src/main/`

| 文件 | 行数 | 职责 |
|---|---|---|
| `main.js` | ~1680 | 应用生命周期、四个窗口的创建/布局、会话与历史、IPC 注册、快捷键、托盘、导出 CSV、置顶守护、冒烟测试驱动 |
| `server.js` | ~440 | HTTP 静态服务 + WebSocket 会话服务：校验、过滤、限流、广播、心跳、优雅关闭 |
| `filter.js` | ~190 | 内容过滤：正则规则 + DFA 词库 + 文本归一化，输出 blocked/flagged |
| `store.js` | ~95 | 设置项存储：SCHEMA 校验、防抖写盘、便携模式路径、退出清理 |
| `network.js` | ~130 | 网卡选择（默认路由优先 + 私有网段加分）、网络位置查询（公用/专用） |
| `topmost.js` | ~63 | Windows 原生调用（koffi）：`SetWindowPos` 置顶加固、`GetAsyncKeyState` 左键轮询 |
| `preload-control.js` | ~37 | 控制面板的桥：暴露 `window.api.*`，只转发白名单方法 |
| `preload-danmaku.js` | ~26 | 弹幕层的桥：收 item/指令/光标/点击，发 hover/modal/ready |
| `preload-cloud.js` | ~13 | 词云页的桥：只读状态 + 历史 |
| `preload-join.js` | ~10 | 投屏引导页的桥：只读状态 + 关闭 |

## 3.2 窗口渲染层 `src/renderer/`

| 目录 | 页面 | 说明 |
|---|---|---|
| `control/` | 控制面板 | 讲师主界面：二维码、房间号、在线数、模式切换、过滤设置、WiFi 信息、最近弹幕、导出 |
| `danmaku/` | 弹幕层 | 全屏透明画布，负责滚动、轨道分配、悬停冻结、图片放大、点击判定 |
| `cloud/` | 词云页 | 全屏不透明，Canvas 画词云 + 右侧消息流（可回溯） |
| `join/` | 投屏引导页 | 全屏不透明，大二维码 + WiFi 账号密码 + 操作指引，给台下的人看 |

其中 `danmaku/layout.js` 是**纯函数模块**（UMD），单独可测；`cloud/words.js` 同理。

### 3.3 观众端 `src/web/`

| 文件 | 说明 |
|---|---|
| `index.html` | 单页结构：状态条 → 现场弹幕流 → 颜色选择 → 输入区 → 操作按钮 |
| `assets/app.js` | 全部交互逻辑：连接/重连、发送、限流倒计时、现场流渲染、主题切换 |
| `assets/style.css` | 深/浅色两套 CSS 变量 + 组件样式 |
| `assets/theme-boot.js` | 防闪烁：在 `<head>` 里同步读出主题并写 `data-theme` |

### 3.4 脚本 `scripts/`

| 脚本 | 用途 |
|---|---|
| `build-win.ps1` | 一键出三种形态：安装版 / 便携版 / 绿色 zip（含镜像变量、图标、目录拷贝） |
| `smoke-test.js` | 冒烟测试：启动真实应用，驱动全部窗口，断言 UI 状态并截图 |
| `test-layout.js` / `test-filter.js` / `test-words.js` | 纯函数单测（轨道算法 / 过滤 / 切词） |
| `probe-click.ps1` | 真实鼠标点击探针：验证「悬停 → 点击 → 大图」链路，dev 与打包版都能跑 |
| `probe-topmost.ps1` | 造一个"激活的置顶全屏窗口"模拟 PPT，验证弹幕层仍在其上 |
| `probe-viewer-feed.js` | 观众端现场流观感验证：灌消息、截图、验证"新消息"提示 |
| `publish-release.js` / `upload-release-asset.ps1` | 发 GitHub Release、上传大附件 |
| `sync-github.ps1` | 网络不稳定时的重试外壳：push + release + 上传一条龙 |
| `make-icons.py` | 由一张源图生成各尺寸 png/ico |

---

## 4. 模块详解

### 4.1 `main.js`：主进程

它不做业务计算，主要是**编排**。按职责可分成七块：

| 职责 | 关键函数 | 说明 |
|---|---|---|
| 窗口管理 | `createOverlay` / `createControlWindow` / `createCloudWindow` / `createJoinWindow` | 四个窗口的创建参数、显示/隐藏、显示器切换 |
| 会话 | `startSession` / `endSession` / `restartSession` / `togglePause` | 起服务、生成 token、结束清理 |
| 历史 | `handleIncoming` / `trimHistory` / `compact` / `publicHistory` | 入历史、裁剪、给渲染层的轻量副本 |
| 状态广播 | `buildState` / `schedulePush` / `sendState` | 把「在线数/暂停/模式/二维码」推给各窗口 |
| IPC | `registerIpc` | 26 个通道，见 §6.3 |
| 系统集成 | `createTray` / `registerShortcuts` / `exportHistory` | 托盘菜单、6 个全局快捷键、CSV 导出 |

| 职责 | 关键函数 | 说明 |
| --- | --- | --- |
| 窗口层级 | `forceTop` 定时器 / `startCursorWatch` | 见 §5.7、§5.8 |

两个容易忽略的设计：

1. **`compact()`**：历史里存原图 dataURL，但推给控制面板/词云的是去掉图片正文的副本，避免每来一张图就把几 MB 的 base64 广播给三个窗口。
2. **`schedulePush()` 合并刷新**：状态变化密集时（如连发消息）用 `setTimeout(0)` 合并成一次推送，避免渲染层高频重排。

### 4.2 `server.js`：会话服务

一个 `http.Server` 同时承担静态资源和 WebSocket 升级（`server.on('upgrade')`），共用 8080 端口——**一个端口，手机只需要记一个地址。**

```
GET  /j/<token>      → 返回 src/web/index.html（带 token 校验）
GET  /assets/*       → 静态资源（缓存头按版本号区分）
GET  /healthz        → 存活探测
WS   /ws?t=<token>   → 会话
```

关键实现：

- **端口探测**：`pickPort(8080)` 依次试 8080…8099，全被占用则 `listen(0)` 交给系统（`canListen` 用临时 socket 预检）。
- **口令校验**：`/j/<token>` 与 `?t=<token>` 都用 `crypto.timingSafeEqual` 比较，避免逐字符试探。
- **心跳**：20s 一次 ping，两次没 pong 就 `terminate()`，防止手机切后台留下僵尸连接。
- **优雅关闭**：`shutdown()` 先广播 `session_end`，再关闭所有连接，最后 `server.close()`。

### 4.3 `filter.js`：内容过滤

三级流水线：

```
原文
 ├─ normalizeLoose()   NFKC 归一化、去零宽字符、全角转半角、去多余空白（宽松，用于命中）
 ├─ normalizeStrict()  再把常见分隔符/符号抹平（严格，用于对抗 "微 信：abc" 这类拆字）
 ├─ RULES（正则）       手机号 / QQ / 微信号 / 网址 / 广告导流
 └─ TRIE（DFA）         内置 48 条通用脏话 + 用户自定义词
        │
        ▼
   { blocked: bool, flagged: bool, category }
```

三档策略由设置 `filterLevel` 控制：

| 档位 | 行为 |
| --- | --- |
| `off` | 完全不过滤 |
| `mark` | 不拦，但标记 `flagged`，控制面板的最近弹幕里显示 ⚠ |
| `block` | 直接拒绝，手机端只收到「内容不合适，换个说法」，**不回显命中词** |

被拦下的消息**不占发送额度**（过滤发生在限流之前），避免误伤后要干等 5 秒。

### 4.4 `store.js`：设置存储

- 白名单 SCHEMA：只允许已知键，逐键做类型/范围/枚举校验，非法值回落到默认值。
- 写盘 300ms 防抖，避免拖动滑块时疯狂写盘。
- **绝不保存消息内容**，只保存设置。
- 便携模式：`portable.txt` 存在或 `PORTABLE_EXECUTABLE_DIR` 时，`userData` 指向 exe 同目录；否则用 `%APPDATA%\local-danmu`。

### 4.5 `network.js`：网卡选择

按分数挑出"最像现场 WiFi"的那块网卡：

| 条件 | 分值 |
| --- | --- |
| 是系统默认路由所在地址 | +200 |
| 名字像 WiFi（WLAN/Wi-Fi/无线） | +30 |
| 名字像有线（Ethernet/以太网） | +20 |
| `192.168.x.x` / `10.x` / `172.16-31.x` | +90 / +70 / +50 |
| 名字像虚拟网卡（VMware/VirtualBox/Hyper-V/Tailscale） | −120 |
| `169.254.x.x`（自动私有地址，通常是没连上） | −80 |
| `100.64-127.x.x`（运营商 NAT / Tailscale） | −40 |

同时用 PowerShell 查 `NetworkCategory`（0=公用），用于提醒"把网络设为专用"，否则 Windows 防火墙会拦手机访问。

### 4.6 `topmost.js`：Windows 原生能力

两个函数，都通过 `koffi`（预编译 FFI，无需编译工具链）直接调 `user32.dll`：

| 函数 | 用途 | 为什么不用 Electron API |
| --- | --- | --- |
| `forceTop(win)` | `SetWindowPos(HWND_TOPMOST, SWP_NOACTIVATE)` | `setAlwaysOnTop(true)` 对**已经置顶**的窗口是空操作，压不住同样置顶且处于激活状态的放映窗口 |
| `isLeftButtonDown()` | `GetAsyncKeyState(VK_LBUTTON)` | 透明窗口反复切换穿透后，Chromium 可能不再给页面派发 click |

koffi 加载失败时会自动降级（置顶退回 `moveTop()`，点击退回 DOM 事件），不会导致应用起不来。

### 4.7 四个渲染层

共同点：**都没有 node 集成**，全部通过 preload 暴露的窄接口与主进程通信（`contextIsolation: true`）。

**弹幕层 `danmaku/`**

- 全屏 `<canvas>`，`requestAnimationFrame` 主循环，每帧：清屏 → 推进 x → 画文字/图片 → 画悬停高亮 → 画大图模态。
- 轨道分配交给 `layout.js`（纯函数，可单测）。
- 交互状态机：`hoverItem`（悬停目标）、`hoverFreeze`（冻结滚动）、`modalItem`（大图）。

- 心跳式接收主进程推来的光标与点击（见 §5.6）。

**控制面板 `control/`**

- 卡片式布局，深浅两套主题（`prefers-color-scheme` + 手动三态）。
- 状态变化用 120ms 防抖合并渲染；二维码由主进程用 `qrcode` 生成 dataURL 后下发。
- 最近弹幕列表带「↻ 重播」按钮，调用 `session:replay`。

**词云页 `cloud/`**

- Canvas 螺旋布局 + 矩形碰撞，词频越高字号越大；词少时自动放大字号填充。
- 右侧消息流可滚动回溯，覆盖的需求是"错过的弹幕能找回来"。

**投屏引导页 `join/`**

- 给台下观众看的大二维码 + WiFi 账号密码 + 三步指引。
- 刻意做成不透明全屏，避免和弹幕层互相干扰。

### 4.8 观众端 H5 `src/web/`

单文件 SPA，无框架、无构建，**首屏 JS 约 20KB**，在微信 X5 内核里也能秒开。

| 能力 | 实现要点 |
| --- | --- |
| 连接 | `new WebSocket('ws://<host>/ws?t=<token>')`，指数退避重连（0.8s 起步，上限 8s，5 次后提示刷新） |
| 发送 | 生成 `msgId`（幂等）、10s 超时、服务端 `ack` 驱动 UI |
| 限流 | 本地倒计时与服务端 `retryAfter` 对齐，按钮禁用 + 读秒 |
| 现场流 | 见 §5.9 |
| 图片 | `<input type="file">` + Canvas 压到长边 800、JPEG 0.82（超 780KB 再降到 0.6），转 dataURL |
| 主题 | `localStorage` 三态，`theme-boot.js` 在 `<head>` 同步应用，避免白闪 |

---

## 5. 关键机制专题

### 5.1 单端口 HTTP + WebSocket

```
const server = http.createServer(handler);        // 静态资源 + /j/<token>
const wss = new WebSocketServer({ noServer: true });
server.on('upgrade', (req, socket, head) => {     // 自己处理升级
  if (!isValidRoom(req.url)) { socket.destroy(); return; }
  wss.handleUpgrade(req, socket, head, ws => wss.emit('connection', ws, req));
});
```

好处：手机只需要一个地址一个端口，扫码链接里不用带第二个端口；也避免了"HTTP 一个端口、WS 另一个端口"时防火墙规则翻倍。

### 5.2 房间口令

- 8 位、去掉易混字符（`abcdefghjkmnpqrstuvwxyz23456789`），约 40 bit 熵。

- 每次「结束培训 → 开始新一场」都会重新生成，旧链接立即失效。
- 校验走恒定时间比较；错误口令直接拒绝 TCP，不返回任何提示页面。

### 5.3 三道限流与幂等

| 级别 | 参数 | 目的 |
| --- | --- | --- |
| 单人 | 两次发送间隔 ≥ `intervalSec`（默认 5s） | 防一个人刷屏 |
| 单 IP | 令牌桶 0.8 条/秒，突发 8 | 防一台设备开多标签/多客户端 |
| 全局 | 令牌桶 4 条/秒，突发 12 | 防全场同时爆发把大屏打爆 |

- 顺序：**过滤 → 单人 → 单 IP → 全局。**被过滤的请求不占任何额度。
- 幂等：客户端生成 `msgId`，服务端按连接缓存最近 40 个，重连重发时返回 `duplicate: true`，不重复上屏。

### 5.4 弹幕轨道算法

核心思路：**不是等上一条完全飞出屏幕，而是等它的尾巴完全进入屏幕**，这样同一条轨道可以连续发车，吞吐高很多。

```
availableAt = now + (width + gap) / speed * 1000
```

- `speed` 的单位是"每秒走过屏宽的百分之几"（默认 8，即 1080p 下约 160 px/s）。
- 一条弹幕可能占多行（图片、或超长文本），所以按"连续 N 行都空闲"查找。
- `reset()` 在清屏、分辨率变化、显示器切换时调用。

### 5.5 渲染循环与交互状态

```
frame()
 ├─ 若 modalItem：只画大图，跳过推进
 ├─ 否则：按 dt 推进 x；出屏的项回收
 ├─ 逐个绘制（文字带描边，保证在浅色 PPT 上也看得清）
 ├─ 若 hoverItem 且未冻结：画薄荷色描边 + 「点击放大」气泡
 └─ 若 modalItem：画遮罩 + 居中大图
```

- **悬停冻结**：鼠标停在图片上 120ms 后才冻结，避免只是路过就把画面定住。
- **命中判定**：图片外扩 16px（`HIT_PAD`）都算命中，手指/触控板不容易点空。

### 5.6 两条点击路径（重要）

Windows 上，透明置顶窗口在"点击穿透 ↔ 接管鼠标"之间反复切换后，**Chromium 偶尔不再把 click 派发给页面**（表现为：开机时能放大，用一阵子就点不动）。

所以点击判定做了两条路，互为备份：

| 路径 | 触发 | 说明 |
| --- | --- | --- |
| A. DOM click | 页面收到 `click` | 正常情况 |
| B. 主进程轮询 | 30Hz `GetAsyncKeyState`，检测到按下边沿就把窗口内坐标发给渲染层 | 页面收不到点击时兜底 |

两条路径进入同一个 `handleClick(x, y)`，用 **180ms 去重**避免"开了又关"。

## 5.7 置顶与穿透

弹幕层的窗口样式（`src/main/main.js` `createOverlay`）：

```
{
  frame: false, transparent: true, backgroundColor: '#00000000',
  focusable: false, skipTaskbar: true, hasShadow: false, roundedCorners: false,
  alwaysOnTop: true, show: false,
  webPreferences: { backgroundThrottling: false }
}
```

显示用 `showInactive()`（不抢焦点），穿透用 `setIgnoreMouseEvents(true, { forward: true })`。

运行时窗口的扩展样式会在这两种状态间切换：

| 状态 | WS_EX_* | 含义 |
| --- | --- | --- |
| 穿透（默认） | `TOPMOST \| NOACTIVATE \| NOREDIRECTIONBITMAP \| LAYERED \| TRANSPARENT` | 点击落到下层应用 |
| 接管（悬停图片/大图） | `TOPMOST \| NOACTIVATE \| NOREDIRECTIONBITMAP` | 点击进我们的窗口 |

**置顶加固**：`forceTop` 每 350ms 对当前应显示的窗口调一次 `SetWindowPos(HWND_TOPMOST, SWP_NOACTIVATE)`。覆盖弹幕层、词云页、引导页、控制面板——否则 WPS/PPT 一进放映，弹幕就沉到下面去了。

## 5.8 光标轮询

`forward: true` 在部分 Windows 机器上并不可靠（实测收不到 `mousemove`），所以悬停判定主要靠主进程 30Hz 轮询 `screen.getCursorScreenPoint()`，把**窗口相对坐标**推给渲染层。

渲染层收到后：

1. 更新 `lastPointer`
2. `imageAt(x, y, 16)` 找悬停目标
3. 有变化才调 `window.dm.hover(bool)`，主进程据此切换穿透状态

## 5.9 观众端现场流

```
server: recentItems（环形缓冲 24 条，图片只存占位）
   ├ 新连接：hello 携带最近 12 条
   └ 每条新消息：broadcast {type:'item'}
观众端：
   ├ 列表 DOM 上限 40 条，超出丢最老
   ├ 原本在底部才自动滚；否则显示「↓ N 条新消息」
   ├ 按 seq 去重（重连时 hello 会重发最近的）
   └ 自己的消息靠右、带薄荷色描边
```

**图片不广播原图**：一张手机照片可能 300KB，100 台就是 30MB 突发。手机端只显示「🖼 图片」占位块。

## 5.10 点赞与快捷反应

两类"轻量信号"，共用一条 WS 通道和一套节流参数，都不计入消息总数、也不进历史正文。

### 点赞

- 观众在手机「现场流」里点别人消息旁的 ♥；自己的消息不显示按钮。

- 服务端按 `seq` 找回消息，先用 `clientId` 去重（同一设备对同一条只算一次），再走节流：250ms 间隔、每人 30 次/分钟、全局 25 次/秒。
- 计数存在 `itemsBySeq`（只留最近 400 条）上，同时同步到 `recentItems` 和主进程 `history`——手机、词云、控制面板看到的是同一份数字。
- 大屏：这条还在屏上时，末尾直接补 `♥N`；已经滚走的，点赞数跨过 5 / 10 / 20 / 50 时重新推一次并打「共鸣」标记，15 秒内只触发一次，讲师可在面板用 `autoReplay` 关掉。

**快捷反应**

- 三种固定语义：`agree` / `confused` / `slow`，手机端三个常驻按钮，2 秒冷却。
- 只做计数：`signals.total` 是累计值，`signals.recent` 是最近 60 秒窗口，控制面板的「现场信号」卡片用条形图展示。
- 不写进弹幕流，所以不会把大屏刷满，也不会挤占消息限流额度。

**词云**

- 词频 = 出现次数 + `log2(1 + 赞数) × 0.8`。用对数是为了让高赞有优势，但不至于一条消息统治全场。
- 词云页右侧新增「共鸣 TOP5」，直接按点赞数排序展示原文。

## 5.11 词云

`words.js` 的切分规则：

1. 短句（≤12 字）整条当一个词——现场短答语义最准
2. 长句按中英文标点/空格切成小段，每段单独计数
3. 去掉句尾语气词（的/了/啊/吧…），让"太难了"和"太难"尽量合并
4. 纯数字/纯符号丢弃

已知局限：**不做同义词合并**（"难"和"太难"算两个词），现场引导大家发短词效果最好。

## 5.12 隐私与落盘

| 数据 | 存放位置 | 生命周期 |
| --- | --- | --- |
| 观众消息 | 主进程内存（`history`） | 结束培训即释放 |
| 图片 | 内存中的 dataURL（最多 20 张） | 同上 |
| 设置项 | `%APPDATA%\local-danmu\settings.json`（便携模式在 exe 旁） | 持久 |
| 会话数据（缓存/Cookie） | `%TEMP%\localdanmu-<pid>` | 退出时删除 |

没有任何外发请求（除了「打开网络设置」「打开仓库」这两个用户主动点击的动作）。

## 5.13 打包与分发

`electron-builder` 一次构建出三种形态：

| 形态 | 产物 | 特点 |
| --- | --- | --- |
| 安装版 | `LocalDanMu-Setup-x.y.z.exe`（NSIS） | 装到 `%LOCALAPPDATA%`，开始菜单快捷方式 |
| 便携版 | `LocalDanMu-Portable-x.y.z.exe` | 单 exe，运行时解压到临时目录 |

| 形态 | 产物 | 特点 |
| --- | --- | --- |
| 绿色版 | `LocalDanMu-x.y.z-win-x64.zip` | 解压即用，启动最快；带 `portable.txt` 时设置写在 exe 旁 |

两个打包细节：

- `asarUnpack: ["**/node_modules/koffi/**"]` ——koffi 是原生模块，必须从 asar 里解出来才能加载。
- `scripts/build-win.ps1` 会设置 `ELECTRON_MIRROR` / `ELECTRON_BUILDER_BINARIES_MIRROR` 指向国内镜像，否则在国内网络下构建会卡在下载 Electron 二进制。

---

# 6. 契约速查

## 6.1 WebSocket 消息（服务端 → 客户端）

| type | 何时 | 主要字段 |
| --- | --- | --- |
| `hello` | 连接建立 | `proto`、`intervalSec`、`paused`、`recent[]`、`signals`、`serverTs` |
| `ack` | 收到一条 message | `msgId`、`status`（ok/invalid/too_fast）、`seq`、`retryAfter`、`reason` |
| `item` | 有消息通过校验 | `seq`、`kind`（text/image）、`text`、`color`、`clientId`、`ts` |
| `config` | 讲师改设置 | `intervalSec` |
| `paused` | 暂停/恢复 | `paused` |
| `session_end` | 结束培训 | `reason` |
| `like` | 有人点赞 | `seq`、`likes`（广播给所有人） |
| `like-reject` | 点赞被拒 | `seq`、`reason`（gone/own/duplicate/too_fast/too_many/busy）、`likes` |
| `react` | 有人点了快捷反应 | `kind`（agree/confused/slow）、`signals`（累计 + 最近 60 秒） |
| `react-reject` | 反应被拒 | `reason`（bad_kind/too_fast/busy） |

## 6.2 WebSocket 消息（客户端 → 服务端）

| type | 字段 |
| --- | --- |
| `message` | `msgId`、`contentType`（text/image）、`payload`、`color`、`clientId`、`ts` |
| `like` | `seq`、`clientId`（同一设备对同一条只算一次） |
| `react` | `kind`（agree/confused/slow） |

## 6.3 IPC 通道（渲染层 ↔ 主进程）

| 通道 | 方向 | 作用 |
| --- | --- | --- |
| `state:get` / `state:push` | 请求 / 推送 | 全量状态（URL、二维码、在线数、模式、显示器…） |
| `session:togglePause` / `session:clear` / `session:end` / `session:restart` | 请求 | 会话控制 |
| `session:replay` | 请求 | 重播某条历史 |

| 通道 | 方向 | 作用 |
| --- | --- | --- |
| `history:get` | 请求 | 取历史（已去图片正文） |
| `app:setConfig` / `app:setDisplay` / `app:setNic` | 请求 | 改设置/投屏显示器/首选网卡 |
| `app:toggleOverlay` / `app:toggleJoin` / `app:setMode` | 请求 | 显隐弹幕层 / 引导页 / 切换词云 |
| `app:copy` / `app:export` / `app:importWords` | 请求 | 复制链接 / 导出 CSV / 导入词表 |
| `app:openRepo` / `app:openNetworkSettings` / `app:refreshNetwork` | 请求 | 打开仓库 / 网络设置 / 刷新网卡 |
| `app:hidePanel` / `app:minimize` / `app:quit` | 请求 | 窗口控制 |
| `dm:item` / `dm:cmd` | 推送 | 新弹幕 / 指令（清屏、暂停、关大图） |
| `dm:cursor` / `dm:click` | 推送 | 光标位置 / 左键按下（兜底） |
| `dm:hover` / `dm:modal` / `dm:ready` | 上报 | 接管鼠标 / 大图模式 / 渲染就绪 |
| `join:close` | 请求 | 关闭投屏引导页 |

## 6.4 设置项

| 键 | 类型/范围 | 默认 | 说明 |
| --- | --- | --- | --- |
| `intervalSec` | 1–60 | 5 | 每人发送间隔 |
| `fontPx` | 20–72 | 38 | 弹幕字号 |
| `speed` | 4–30 | 8 | 每秒屏宽百分比 |
| `area` | full/top/middle/bottom | full | 弹幕可用区域 |
| `opacity` | 30–100 | 100 | 弹幕不透明度 |
| `displayId` | any | null | 投屏显示器 |
| `preferredIp` | any | null | 首选网卡地址 |
| `theme` | system/dark/light | system | 界面主题 |
| `wifiSsid` / `wifiPassword` | string | '' | 现场 WiFi（显示在引导页） |
| `mode` | scroll/cloud | scroll | 投影模式 |
| `filterLevel` | off/mark/block | block | 过滤强度 |
| `autoReplay` | bool | true | 高赞弹幕是否自动重新上屏 |
| `customWords` | string ≤6000 | '' | 自定义词表（换行分隔） |

# 7. 启动时序

```
app.whenReady()
 ├─ 1. 设置 sessionData 到 %TEMP%\localdanmu-<pid>
 ├─ 2. 便携模式判定（portable.txt / PORTABLE_EXECUTABLE_DIR）
 ├─ 3. createControlWindow()      控制面板先出来，讲师有东西可看
 ├─ 4. createOverlay()            弹幕层（隐藏，等 ready-to-show 再 showInactive）
 ├─ 5. startCursorWatch()         30Hz 光标 + 左键轮询
 ├─ 6. createTray() / registerShortcuts()
 ├─ 7. startSession()
 │     ├─ pickPort(8080)          8080…8099，全占则系统分配
 │     ├─ randomToken(8)          房间口令
 │     ├─ network.pick()          选网卡
 │     ├─ bridge.listen(port)     起 HTTP + WS
 │     └─ 生成二维码 dataURL
 ├─ 8. applyMode()                滚动/词云
 └─ 9. projectTo(display)         投到指定显示器（跟随显示器变化重建窗口）
```

退出时：`before-quit` → 停服务 → 清 `%TEMP%` 会话目录 → 移除快捷键/托盘。

# 8. 需求 → 实现对照

| 需求方案条目 | 实现位置 | 验证方式 |
|---|---|---|
| 弹幕不遮挡 PPT | `main.js createOverlay` + `topmost.js forceTop` | `probe-topmost.ps1`（模拟置顶全屏窗口） |
| 不挡鼠标操作 | `setIgnoreMouseEvents(true)` + 悬停才接管 | `probe-click.ps1` |
| 观众不装 App | `src/web` 纯 H5，单端口 | 微信扫码实测 |
| 数据不出局域网 | 无外发请求，`server.js` 只监听内网 | 抓包 / 代码审查 |
| 默认不落盘 | 消息仅在内存；设置项白名单 | `store.js` 审查 |
| 仅 Windows | 使用 Win32 API（koffi） | — |
| 显示器选择 | `app:setDisplay` + `applyDisplayToOverlay` | 冒烟 `displays` 断言 |
| 房间口令 | `randomToken` + `timingSafeEqual` | 冒烟 `newToken` 断言 |
| 微信扫码 | `qrcode` 生成 + 引导页 | 真机 |
| 不做审核后上屏 | 无审核队列，校验通过即上屏 | — |
| 弹幕速度过快 | `speed` 语义改为"每秒屏宽百分比"，范围 4–30 | `test-layout.js` |
| 错过的弹幕能回溯 | 控制面板「最近弹幕」+ 词云页右侧列表 + `session:replay` | 冒烟 `controlLogShot` |
| 词云看比例 | `cloud/words.js` + Canvas 布局 | `test-words.js`、冒烟 `cloud` |
| 文字有颜色区别 | 8 色白名单，观众自选/随机 | 冒烟颜色断言 |
| 图片点击不灵敏 | 16px 外扩命中 + 两条点击路径 + 180ms 去重 | `probe-click.ps1` |
| 关键词过滤词典 | `filter.js`：5 条正则 + 48 词 + 自定义/导入 | `test-filter.js`（13 例） |
| 增强：点赞 / 快捷反应 | `server.js` 的 `handleLike` / `handleReact`、`danmaku.js` 的 ♥N、控制面板「现场信号」 | 冒烟 `signals` / `likeAfter` 断言 |
| 版权信息 | 控制面板头部 + 引导页/词云页底部 | 截图 |
| MyPassword 风格 | 深墨绿 + 薄荷配色 token，深浅两套 | 截图 |

# 9. 测试与验证

| 层次 | 手段 | 覆盖 |
| --- | --- | --- |
| 纯函数单测 | `test-layout.js`（10）/ `test-filter.js`（13）/ `test-words.js`（9） | 轨道算法、过滤规则、切词 |
| 冒烟测试 | `smoke-test.js`（26 断言） | 起真实应用，驱动控制面板、弹幕层、大图、词云、引导页、观众端、生命周期，并截图存档 |
| 真实输入探针 | `probe-click.ps1` / `probe-topmost.ps1` / `probe-viewer-feed.js` | 鼠标点击、置顶层级、现场流 |
| 发布脚本 | `publish-release.js` + `upload-release-asset.ps1` | Release 文案与三附件 |

> 经验：**冒烟测试里的 `sendInputEvent` 会绕过系统层**，之前"图片点击"一直显示通过，实际真机不可用。凡是涉及"鼠标能不能点到"的，必须用 `probe-click.ps1` 这种真实鼠标事件验证。

# 10. 已知限制与踩坑记录

| 现象 | 根因 | 处理 |
| --- | --- | --- |
| 图片悬停正常但点不开 | 透明窗口反复切换穿透后 Chromium 不再派发 click | 主进程轮询左键兜底（§5.6） |
| 悬停完全没反应 | `setIgnoreMouseEvents({forward:true})` 的 mousemove 不可靠 | 改为 30Hz 光标轮询（§5.8） |
| WPS/PPT 放映时弹幕被盖住 | `setAlwaysOnTop(true)` 对已置顶窗口是空操作 | `SetWindowPos(HWND_TOPMOST)` 每 350ms（§5.7） |
| 打包后 koffi 加载失败 | 原生模块被塞进 asar | `asarUnpack` |
| `ELECTRON_RUN_AS_NODE=1` 时双击没反应 | Electron 退化成 Node | 启动前清该环境变量 |
| `package.json` 解析失败 | PowerShell `Set-Content -Encoding UTF8` 写了 BOM | 改用 Node 写文件；`git ls-files` 扫 BOM |
| Release 正文中文变问号 | Windows PowerShell 以 ASCII 发送 JSON body | 改用 Node 发请求（UTF-8） |
| 国内构建卡在下载 | electron-builder 直连 GitHub | 设置 `ELECTRON_MIRROR` 等镜像变量 |

其它已知限制：

- 词云不做同义词合并。
- 过滤不内置政治敏感词，需要用户自行导入词表。
- 图片悬停时那一小块区域点不到下层的 PPT。
- 投屏引导页会公开 WiFi 密码（设计如此，方便台下扫码）。
- 观众端现场流的图片只有占位，没有缩略图。

# 11. 其它技术路线探讨（简要）

| 路线 | 优点 | 代价 | 适用判断 |
| --- | --- | --- | --- |
| **Electron（当前）** | 一套 JS 搞定四个窗口；透明窗口 + 置顶 + 画布性能足够；调试方便 | 安装包 ~110MB，内存 ~300MB | 讲师机是普通办公本时完全够用 |
| **Tauri 2** | 包体可小到 10MB 级，WebView2 复用系统组件 | 透明/置顶/穿透在 Windows 上的坑要自己踩一遍；Rust 门槛 | 如果对安装包体积敏感可以换 |
| **WPF / WinUI 3** | 原生、启动快、Win32 能力最直接 | 前端生态弱，二维码/词云要自己实现；开发效率低 | 长期维护、功能稳定的企业内部工具 |
| **原生 C# 悬浮窗 + 浏览器观众端** | 悬浮层用 Win32 最稳，观众端仍是 H5 | 两套技术栈，两个开发者 | 团队里有 .NET 同学时值得考虑 |
| **纯 Web（讲师机开浏览器全屏）** | 零安装、最轻 | 浏览器无法做到"点击穿透 + 置顶在 PPT 之上"，**不满足硬约束** | 不可行 |
| **OBS 插件 / 直播推流** | 有现成的图层合成 | 要装 OBS、配置复杂，讲师学习成本高 | 不推荐给非技术讲师 |
| **浏览器扩展** | 可直接叠在网页上 | 只能覆盖浏览器内容，PPT 放映窗口管不到 | 不适用 |
| **WebRTC 直连** | 无需服务器，理论最低延迟 | 现场局域网信令复杂、NAT/防火墙不可控，收益为零（本来就在同一网段） | 过度设计 |

一句话总结：**本项目真正的难点不在 Web 技术，而在 Windows 的窗口层级与输入穿透。**换任何技术栈，这部分都要重新解决一遍；Electron 至少让我们用一套语言同时解决了 UI 和窗口问题。

# 12. 扩展点

| 想加的能力 | 建议落点 | 注意 |
| --- | --- | --- |
| 审核后上屏 | `server.js` 校验后进待审队列，控制面板加"通过/驳回" | 会改变"即时反馈"的体验，建议做成开关 |
| 消息持久化 | `handleIncoming` 处落 SQLite，配合"导出" | 与"默认不落盘"冲突，必须显式开启 |
| 多会场 | 一个进程多 `bridge` 实例，token 前缀区分 | 端口复用要重新设计 |
| 观众昵称/匿名 | 消息体加 `nick`，服务端校验长度 | 会带来重名与冒名问题 |
| 现场投票 | 复用词云的聚合能力，加"单选题"消息类型 | 需要新的 UI，不是小改动 |
| 大屏录像 | `capturePage` 定时抓帧 + ffmpeg 合成 | CPU 开销大，慎用 |

*文档随代码演进，改动较大时请同步更新 `docs/architecture.md` 与 `CHANGELOG`。*